JavaScript Tutorial

A Complete Guide

15 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/Javascript/

CHAPTER 1

Getting Started with JavaScript

What is JavaScript?

JavaScript is a high-level, interpreted programming language that makes web pages interactive. It runs in the browser and is one of the three core technologies of the web alongside HTML and CSS.

Where to Put JavaScript

Inline JavaScript

You can add JavaScript directly inside HTML elements using event attributes:

```
<button onclick="alert('Hello!')">Click Me</button>
```

Internal JavaScript

Place code inside a `<script>` tag in the `<head>` or before the closing `</body>`:

```
<script>
  console.log("Hello from internal JS");
</script>
```

External JavaScript

Link a separate `.js` file using the `src` attribute:

```
<script src="app.js"></script>
```

Output Methods

JavaScript provides several ways to display output:

```
// Console output — great for debugging
console.log("This goes to the browser console");

// Write directly to the document
document.write("This appears on the page");

// Show a popup dialog
alert("This is an alert box");
```

Basic Syntax

JavaScript statements end with a semicolon (optional but recommended). The language ignores whitespace, making it flexible in formatting.

Comments

```
// This is a single-line comment

/*
  This is
  a multi-line
  comment
*/
```

Variables

Use `var`, `let`, or `const` to declare variables:

```
// var — function-scoped, avoid in modern code
var name = "Alice";

// let — block-scoped, can be reassigned
let age = 25;
age = 26;

// const — block-scoped, cannot be reassigned
const birthYear = 1998;
```

> **Tip:** Prefer `const` by default and use `let` only when you need to reassign. Avoid `var` in modern JavaScript.

Operators

Arithmetic Operators

```
let sum = 10 + 5;     // 15
let diff = 10 - 5;    // 5
let product = 10 * 5; // 50
let quotient = 10 / 5; // 2
let mod = 10 % 3;     // 1
let exp = 2 ** 3;     // 8
```

Assignment Operators

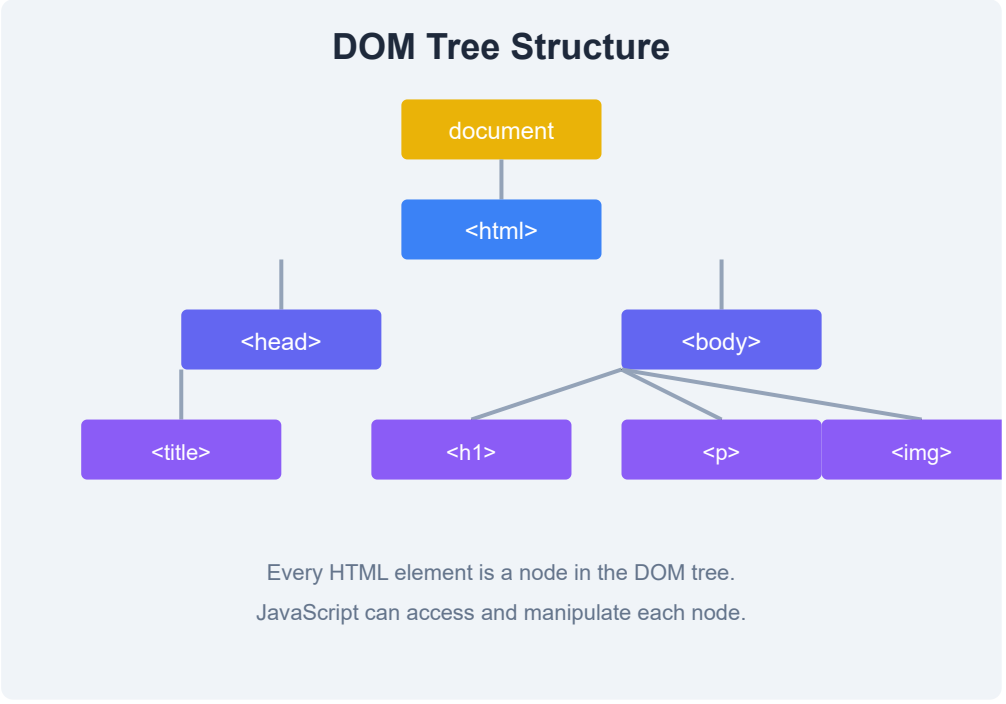
```
let x = 10;
x += 5;  // x = 15
x -= 3;  // x = 12
x *= 2;  // x = 24
x /= 4;  // x = 6
```

Comparison Operators

```
5 == "5"   // true  (loose equality)
5 === "5"  // false (strict equality)
5 != "5"   // false
5 !== "5"  // true
5 > 3     // true
5 <= 5    // true
```

Logical Operators

```
let a = true, b = false;
a && b  // false (AND)
a || b  // true  (OR)
!a      // false (NOT)
```



> **Key Point:** JavaScript is case-sensitive. `myVariable` and `myvariable` are two different identifiers. Use camelCase for variable names by convention.

Exercise: Create a script that declares two variables `firstName` and `lastName`, combines them into a full name using the `+` operator, and logs the result to the console. Then use `alert()` to display the full name.

CHAPTER 2

Control Flow

Control flow determines the order in which your code executes. JavaScript provides conditionals, loops, and iteration tools to control program execution.

Conditional Statements

if / else if / else

```
let score = 85;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 75) {
  console.log("Grade: B");
} else if (score >= 60) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

Switch Statement

Use `switch` when comparing a single value against many possible cases:

```
let day = 3;
let dayName;

switch (day) {
  case 1:
    dayName = "Monday";
    break;
  case 2:
    dayName = "Tuesday";
    break;
  case 3:
    dayName = "Wednesday";
    break;
  case 4:
    dayName = "Thursday";
    break;
  case 5:
    dayName = "Friday";
    break;
  default:
    dayName = "Weekend";
}

console.log(dayName); // "Wednesday"
```

> **Remember:** Always include `break` in each case unless you intentionally want fall-through. The `default` clause runs when no case matches.

Ternary Operator

A shorthand for simple if/else conditions:

```
let age = 20;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status); // "Adult"
```

Loops

for Loop

```
for (let i = 0; i < 5; i++) {
  console.log("Iteration:", i);
}
// Prints 0, 1, 2, 3, 4
```

while Loop

```
let count = 0;
while (count < 5) {
  console.log(count);
  count++;
}
```

do-while Loop

Executes the body at least once, then checks the condition:

```
let x = 0;
do {
  console.log(x);
  x++;
} while (x < 3);
```

Iteration Methods

for...of

Iterates over iterable values (arrays, strings, etc.):

```
let colors = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color);
}
```

for...in

Iterates over enumerable property keys of an object:

```
let person = { name: "Alice", age: 30 };
for (let key in person) {
  console.log(key + ":", person[key]);
}
```

Jump Statements

`break` exits a loop entirely. `continue` skips the current iteration and moves to the next one.

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // Skip 3
  if (i === 7) break;    // Stop at 7
  console.log(i);
}
// Prints: 0, 1, 2, 4, 5, 6
```

Best Practice: Use `for...of` for arrays and `for...in` for objects. Avoid `for...in` on arrays since it iterates over keys (indices as strings) and may include inherited properties.

Exercise: Write a loop that prints all even numbers from 1 to 20. Then convert the same logic into a `while` loop. Finally, create a `switch` statement that maps a numeric month (1–12) to its name and logs it.

CHAPTER 3

Working with Data

JavaScript provides built-in types and objects for working with strings, numbers, dates, and more. Mastering these is essential for real-world development.

Strings

Strings represent text and are enclosed in single quotes, double quotes, or backticks.

```
let single = 'Hello';
let double = "World";
let template = `Hello, ${single}!`; // Template literal with interpolation
```

String Methods

```
let str = "JavaScript is awesome";

console.log(str.length);            // 21
console.log(str.toUpperCase());     // "JAVASCRIPT IS AWESOME"
console.log(str.toLowerCase());     // "javascript is awesome"
console.log(str.indexOf("is"));     // 11
console.log(str.slice(0, 10));      // "JavaScript"
console.log(str.includes("awesome")); // true
console.log(str.split(" "));        // ["JavaScript", "is", "awesome"]
console.log(str.replace("awesome", "fun")); // "JavaScript is fun"
```

Template Literals

Template literals (backticks) allow embedded expressions and multi-line strings:

```
let name = "Bob";
let age = 30;
let greeting = `Hi, I'm ${name} and I'm ${age} years old.`;

let multiLine = `
  This is
  a multi-line
  string
`;
```

Numbers

JavaScript has a single `Number` type for both integers and floating-point numbers.

```
let integer = 42;
let float = 3.14;
let negative = -10;

// Parsing strings to numbers
console.log(parseInt("42px"));    // 42
console.log(parseFloat("3.14em")); // 3.14

// Formatting
let num = 123.456;
console.log(num.toFixed(2));      // "123.46"
console.log(num.toPrecision(4));  // "123.5"

// Special values
console.log(Infinity);            // Infinity
console.log(NaN);                 // NaN (Not a Number)
console.log(isNaN("hello"));      // true
```

Dates

The `Date` object handles dates and times:

```
let now = new Date();
let specific = new Date(2025, 0, 15); // Jan 15, 2025 (months are 0-indexed)
let fromString = new Date("2025-01-15");

console.log(now.getFullYear());
console.log(now.getMonth());      // 0–11
console.log(now.getDate());
console.log(now.getDay());        // 0 (Sunday) – 6
console.log(now.toLocaleDateString("en-US"));
console.log(now.toISOString());
```

> **Note:** Months in JavaScript's Date object are 0-indexed (0 = January, 11 = December). Always double-check when constructing dates.

Temporal API (Modern Alternative)

The Temporal API is a modern replacement for the Date object, providing immutable date/time types:

```
// Temporal is available in modern environments
// const today = Temporal.Now.plainDateISO();
// const dueDate = today.add({ days: 7 });
```

Math Object

```
console.log(Math.PI);           // 3.141592653589793
console.log(Math.round(4.7));   // 5
console.log(Math.floor(4.7));   // 4
console.log(Math.ceil(4.3));    // 5
console.log(Math.random());     // Random number between 0 and 1
console.log(Math.max(1, 5, 3)); // 5
console.log(Math.min(1, 5, 3)); // 1
```

Regular Expressions

RegExp is used for pattern matching in strings:

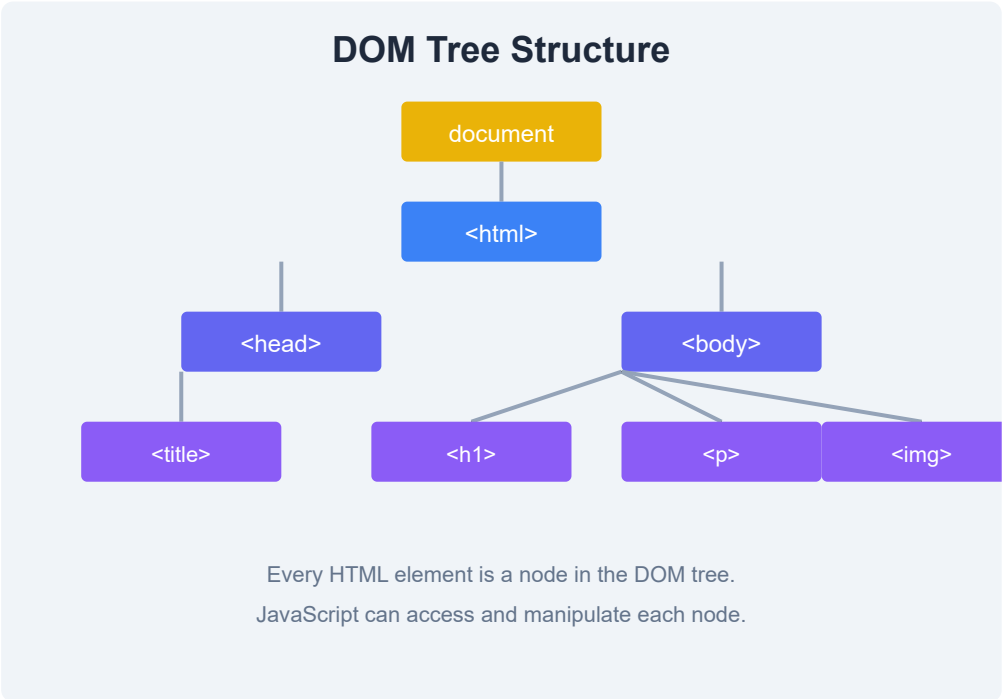
```
let pattern = /hello/i; // Case-insensitive
let text = "Hello World";

console.log(pattern.test(text));       // true
console.log(text.match(pattern));      // ["Hello"]
console.log(text.replace(/world/i, "JS")); // "Hello JS"
```

Type Conversion

```
// Implicit conversion
console.log("5" + 3);   // "53" (string concatenation)
console.log("5" - 3);   // 2 (numeric subtraction)

// Explicit conversion
console.log(Number("42"));    // 42
console.log(String(42));      // "42"
console.log(Boolean(0));      // false
console.log(Boolean(""));     // false
console.log(Boolean("text")); // true
```



Important: JavaScript uses dynamic typing — a variable can hold any type. Use `typeof` to check the type: `typeof 42 // "number"`. Be careful with implicit coercion; it often leads to bugs.

Exercise: Create a script that takes a date string, converts it to a Date object, and formats it as "Month Day, Year" (e.g., "January 15, 2025"). Then generate a random integer between 1 and 100 using `Math.random()` and `Math.floor()`. Finally, use a template literal to display both results.

CHAPTER 4

Functions & Objects

Functions are reusable blocks of code. Objects store collections of key-value pairs. Together they form the backbone of JavaScript programs.

Function Declarations

A function declaration defines a named function that is hoisted to the top of its scope:

```
function greet(name) {
  return "Hello, " + name + "!";
}

console.log(greet("Alice")); // "Hello, Alice!"
```

Function Expressions

A function expression assigns a function to a variable. It is not hoisted:

```
const greet = function(name) {
  return "Hello, " + name + "!";
};

console.log(greet("Bob")); // "Hello, Bob!"
```

Arrow Functions

Arrow functions provide a concise syntax and do not have their own `this` binding:

```
const greet = (name) => {
  return `Hello, ${name}!`;
};

// Even shorter with implicit return
const greetShort = (name) => `Hello, ${name}!`;

console.log(greet("Charlie")); // "Hello, Charlie!"
```

Parameters and Arguments

Functions can have default parameters and rest parameters:

```
function multiply(a, b = 1) {
  return a * b;
}

function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(multiply(5));      // 5 (b defaults to 1)
console.log(sum(1, 2, 3, 4));  // 10
```

> **Tip:** Arrow functions are ideal for short callbacks and methods that should not rebind `this`. Use function declarations for top-level functions that need hoisting.

Object Literals

Objects store data as key-value pairs using curly braces:

```
const person = {
  firstName: "Alice",
  lastName: "Johnson",
  age: 30,
  greet() {
    console.log(`Hi, I'm ${this.firstName}`);
  }
};

// Accessing properties
console.log(person.firstName); // "Alice"
console.log(person["lastName"]); // "Johnson"

person.greet(); // "Hi, I'm Alice"
```

Dot vs Bracket Notation

Use dot notation for known property names. Use bracket notation for dynamic keys or names with special characters:

```
const key = "age";
console.log(person[key]); // 30

const product = { "product-name": "Widget" };
console.log(product["product-name"]); // "Widget"
```

The `this` Keyword

`this` refers to the object that owns the executing code:

```
const car = {
  brand: "Toyota",
  describe() {
    console.log(`This is a ${this.brand}`);
  }
};

car.describe(); // "This is a Toyota"
```

Constructor Functions

Constructor functions create objects with shared structure (pre-ES6 pattern):

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

const alice = new Person("Alice", 30);
console.log(alice.name); // "Alice"
```

ES6 Classes

Classes provide a cleaner syntax for constructor functions and inheritance:

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  speak() {
    console.log(`${this.name} says ${this.sound}`);
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name, "Woof");
  }
}

const dog = new Dog("Rex");
dog.speak(); // "Rex says Woof"
```

Scope

JavaScript has three levels of scope:

```
// Global scope
let globalVar = "I'm global";

function test() {
  // Function scope
  let funcVar = "I'm local to this function";

  if (true) {
    // Block scope
    let blockVar = "I'm local to this block";
    console.log(globalVar); // Accessible
    console.log(funcVar);   // Accessible
  }
  // console.log(blockVar); // ReferenceError
}
```

Closures

A closure is a function that remembers its outer variables even after the outer function has returned:

```
function createCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Key Insight: Closures are powerful for data privacy and creating factory functions. Each call to `createCounter()` creates an independent closure with its own `count`.

Exercise: Write a `createGreeter` function that takes a greeting word and returns a new function. The returned function should accept a name and log the greeting followed by the name. Then use it to create a "Hello" greeter and a "Goodbye" greeter. Also write a `Rectangle` class with `width`, `height`, and an `area()` method.

CHAPTER 5

Error Handling & Debugging

Errors are inevitable in programming. JavaScript provides robust tools for catching, handling, and debugging errors effectively.

try / catch / finally

The `try` block contains code that may throw an error. The `catch` block handles it. The `finally` block always runs:

```
try {
  let result = riskyOperation();
  console.log(result);
} catch (error) {
  console.error("Something went wrong:", error.message);
} finally {
  console.log("This always runs — cleanup code here");
}
```

> **Best Practice:** Always use `try/catch` around code that can fail — like network requests, file operations, or JSON parsing. The `finally` block is perfect for cleanup (closing connections, hiding spinners).

Throwing Errors

Use `throw` to create custom errors:

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed");
  }
  if (typeof a !== "number" || typeof b !== "number") {
    throw new TypeError("Both arguments must be numbers");
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (err) {
  console.error(err.name + ":", err.message);
}
```

Error Types

JavaScript has several built-in error constructors:

```
// Standard errors you'll encounter:
new Error("Generic error");
new SyntaxError("Invalid syntax");
new ReferenceError("Variable not defined");
new TypeError("Wrong type");
new RangeError("Value out of range");
new URIError("Invalid URI");
```

The `debugger` Statement

Insert `debugger;` in your code to pause execution and inspect variables in the browser's developer tools:

```
function calculateTotal(items) {
  let total = 0;
  for (let item of items) {
    debugger; // Execution pauses here
    total += item.price;
  }
  return total;
}
```

Console Methods

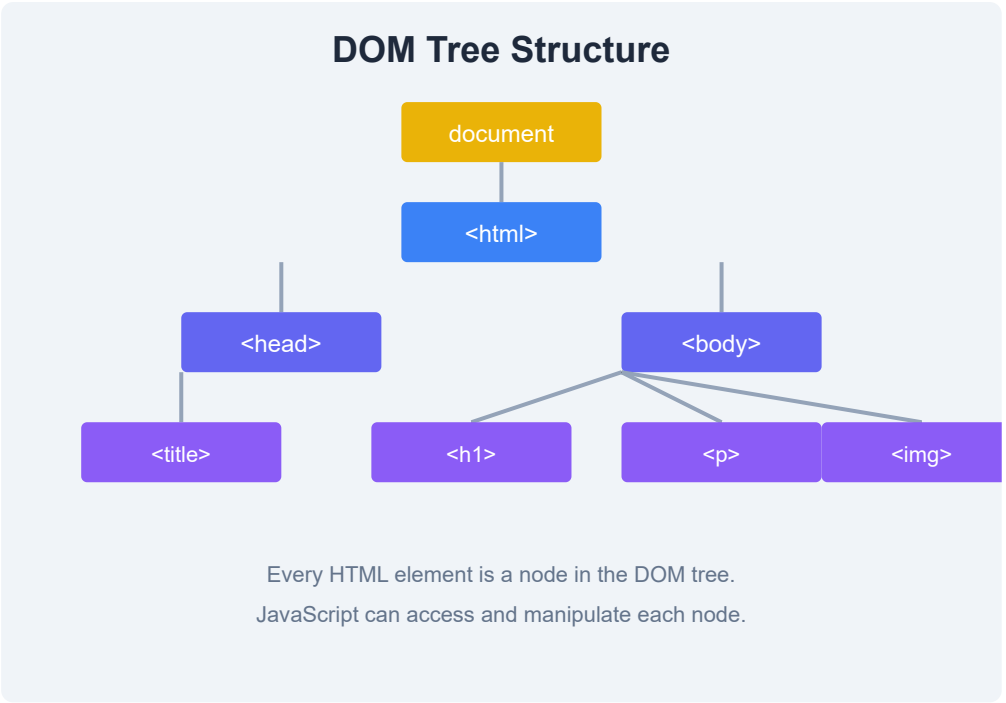
The console offers much more than just `log`:

```
console.log("Standard log message");
console.info("Informational message");
console.warn("Warning — something might be wrong");
console.error("Error — something definitely went wrong");

// Tabular output for arrays of objects
const users = [
  { name: "Alice", age: 30 },
  { name: "Bob", age: 25 }
];
console.table(users);

// Grouping related logs
console.group("User Details");
console.log("Name: Alice");
console.log("Age: 30");
console.groupEnd();

// Timing operations
console.time("loop");
for (let i = 0; i < 1000000; i++) {}
console.timeEnd("loop");
```



Browser DevTools

Modern browsers include powerful developer tools. Key panels include:

- **Console** — View log messages and interact with JavaScript
- **Sources** — Set breakpoints, step through code, inspect scopes
- **Network** — Monitor HTTP requests and responses
- **Elements** — Inspect and modify the DOM

Press `F12` or `Ctrl+Shift+I` to open DevTools in most browsers.

JavaScript Style Guide

Consistent code style improves readability. Common conventions include:

```
// Use 2 spaces for indentation
// Use semicolons consistently
// Use camelCase for variables and functions
// Use PascalCase for classes
// Use descriptive names
const MAX_SIZE = 100; // Constants in UPPER_SNAKE_CASE

function fetchUserData(userId) {
  // ...
}
```

Strict Mode

Enable strict mode with `"use strict";` at the top of a script or function. It catches common mistakes and prevents unsafe actions:

```
"use strict";

// These would throw errors in strict mode:
// x = 10;           // ReferenceError (no var/let/const)
// delete Object.prototype; // TypeError (cannot delete)
```

> **Tip:** ES6 modules and classes are automatically in strict mode. Always write new code in strict mode to avoid common pitfalls.

JavaScript Versions and Compatibility

JavaScript follows the ECMAScript (ES) specification:

- **ES5 (2009)** — Widely supported, `strict mode`
- **ES6 / ES2015** — Major update: `let`, `const`, arrow functions, classes, modules
- **ES2016+** — Annual releases with incremental features
- **ES2024** — Latest features including well-formed Unicode

Use **Babel** or **TypeScript** to transpile modern JS for older browser support.

Exercise: Write a function `safeJSONParse(str)` that attempts to parse a JSON string inside a `try/catch`. If parsing fails, log a descriptive error and return `null`. Test it with valid JSON (`'{"name":"Alice"}'`) and invalid JSON (`'not json'`). Also add a `console.table` showing at least three test cases with their results.

CHAPTER 6

DOM & Events

The Document Object Model (DOM) is JavaScript's interface to HTML. It lets you read, modify, and react to every element on the page.

What is the DOM?

The DOM represents an HTML document as a tree of objects. Each HTML element is a *node* that can be accessed and manipulated with JavaScript. The `document` object is the root of this tree.

The `document` Object

The `document` object provides access to the entire page:

```
console.log(document.title);       // Page title
console.log(document.URL);         // Current URL
console.log(document.body);        // <body> element
console.log(document.documentElement); // <html> element
```

Selecting Elements

getElementById

```
const header = document.getElementById("main-header");
```

querySelector and querySelectorAll

Use CSS-style selectors for flexible element selection:

```
// Returns the first match
const firstButton = document.querySelector(".btn");

// Returns all matches as a NodeList
const allButtons = document.querySelectorAll(".btn");

// Complex selectors
const navLinks = document.querySelectorAll("nav a.active");
```

> **Performance:** `getElementById` is faster than `querySelector`, but `querySelector` is more flexible. Use whichever is more readable for your use case.

Traversing the DOM

Move between nodes once you have a reference:

```
const parent = element.parentNode;
const children = element.children;       // HTMLCollection
const firstChild = element.firstElementChild;
const lastChild = element.lastElementChild;
const nextSibling = element.nextElementSibling;
const prevSibling = element.previousElementSibling;
```

Modifying Content

```
const el = document.querySelector("#output");

// Change text (safer — prevents XSS)
el.textContent = "New text content";

// Change HTML (use with caution)
el.innerHTML = "<strong>Bold text</strong>";

// Change attributes
el.setAttribute("class", "highlight");
console.log(el.getAttribute("class"));

// Work with classes
el.classList.add("active");
el.classList.remove("inactive");
el.classList.toggle("visible");
console.log(el.classList.contains("active"));
```

Creating and Removing Elements

```
// Create a new element
const newDiv = document.createElement("div");
newDiv.textContent = "I was created by JavaScript!";
newDiv.classList.add("box");

// Insert it into the DOM
document.body.appendChild(newDiv);

// Insert at a specific position
const reference = document.querySelector("#container");
reference.insertBefore(newDiv, reference.firstChild);

// Remove an element
const oldEl = document.querySelector(".old");
oldEl.remove(); // Modern approach
// oldEl.parentNode.removeChild(oldEl); // Legacy approach
```

Event Listeners

React to user interactions with `addEventListener`:

```
const button = document.querySelector("#myButton");

button.addEventListener("click", function(event) {
  console.log("Button clicked!", event);
});

// Common events: click, mouseover, mouseout, keydown, keyup,
// submit, focus, blur, change, scroll, resize
```

The Event Object

The event object provides details about the event:

```
document.querySelector("a").addEventListener("click", function(e) {
  e.preventDefault(); // Stop default behavior (navigation)
  console.log("Target:", e.target);
  console.log("Type:", e.type);
  console.log("Coordinates:", e.clientX, e.clientY);
});
```

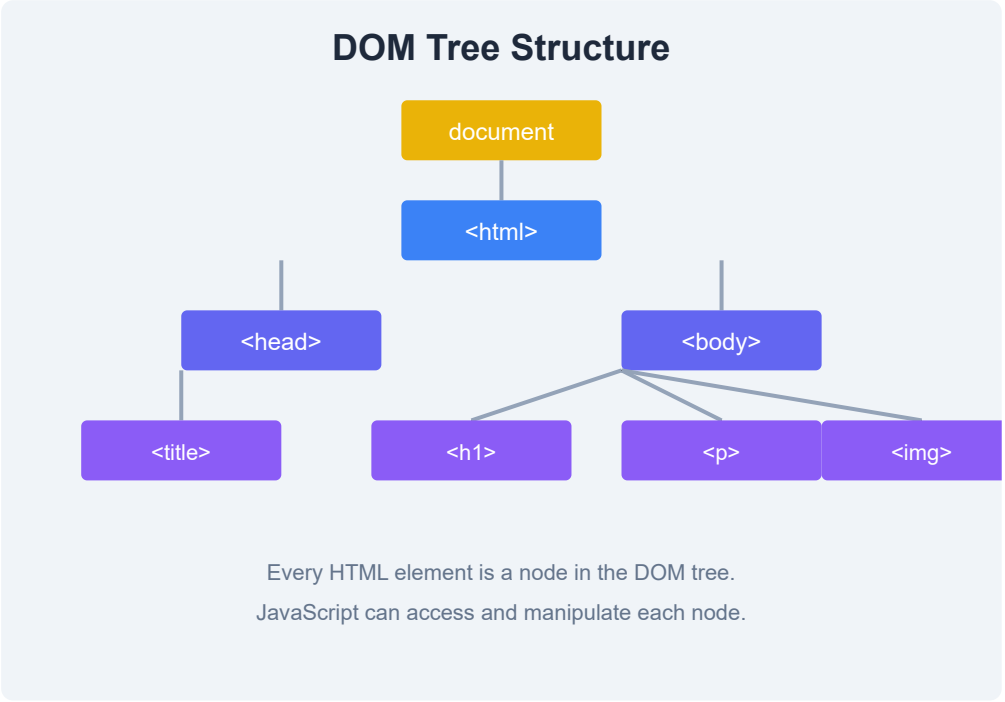
Event Propagation

Events flow through the DOM in three phases:

- **Capturing phase** — Event travels from `document` down to the target
- **Target phase** — Event reaches the target element
- **Bubbling phase** — Event bubbles back up from target to `document`

```
// Stop propagation
element.addEventListener("click", function(e) {
  e.stopPropagation(); // Prevents bubbling
});

// Use capturing phase instead of bubbling
element.addEventListener("click", handler, true);
// Third parameter: true = capture, false (default) = bubble
```



Pro Tip: `e.stopPropagation()` stops the event from moving further in the propagation chain. `e.stopImmediatePropagation()` also prevents other listeners on the same element from firing.

Exercise: Create a simple to-do list with an input field and a button. When the user clicks the button, read the input value, create a new `<li>` element, add it to a `<ul>`, and clear the input. Add a click event on each `<li>` that toggles a `completed` class (which strikes through the text).

CHAPTER 7

Advanced Concepts

This chapter explores advanced JavaScript features including asynchronous programming patterns, modules, metaprogramming with Proxy and Reflect, and typed arrays for binary data.

Asynchronous Programming

JavaScript is single-threaded but non-blocking. Asynchronous operations (like network requests or timers) run in the background and execute callbacks when complete. The event loop manages this concurrency model.

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout callback");
}, 1000);

console.log("End");
// Output: Start, End, Timeout callback
```

We cover asynchronous programming in depth in Chapter 8: Asynchronous JavaScript.

Modules (export / import)

ES6 modules let you split code into separate files. Use `export` to expose values and `import` to consume them:

Named Exports

```
// 📁 math.js
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}
export class Calculator { /* ... */ }
```

```
// 📁 app.js
import { PI, add } from "./math.js";
console.log(add(PI, 2)); // 5.14159
```

Default Exports

```
// 📁 utils.js
export default function formatDate(date) {
  return date.toISOString();
}

// 📁 app.js
import formatDate from "./utils.js";
```

Dynamic Imports

Load modules on demand with `import()`:

```
// Dynamic import returns a promise
button.addEventListener("click", async () => {
  const module = await import("./heavy-module.js");
  module.run();
});
```

> **Important:** Modules are always in strict mode and are deferred by default. Use `<script type="module">` in HTML to load module scripts. Module code runs after the DOM is ready.

Metaprogramming with Proxy

A `Proxy` wraps an object and lets you intercept fundamental operations (getting, setting, deleting properties, etc.) via *handler traps*:

```
const target = { message: "Hello" };

const handler = {
  get(obj, prop) {
    if (prop === "message") {
      return obj[prop] + " (intercepted)";
    }
    return obj[prop];
  },
  set(obj, prop, value) {
    console.log(`Setting ${prop} to ${value}`);
    obj[prop] = value;
    return true; // Indicate success
  }
};

const proxy = new Proxy(target, handler);
console.log(proxy.message); // "Hello (intercepted)"
proxy.newProp = 42;         // Logs: "Setting newProp to 42"
```

Metaprogramming with Reflect

`Reflect` provides methods that correspond to each Proxy trap. Use `Reflect` inside proxy handlers for default behavior:

```
const handler = {
  get(obj, prop, receiver) {
    console.log(`Getting ${prop}`);
    return Reflect.get(obj, prop, receiver);
  },
  set(obj, prop, value, receiver) {
    console.log(`Setting ${prop} = ${value}`);
    return Reflect.set(obj, prop, value, receiver);
  }
};

const proxy = new Proxy({}, handler);
proxy.name = "Alice"; // Logs: Setting name = Alice
console.log(proxy.name); // Logs: Getting name → "Alice"
```

Typed Arrays

Typed arrays provide views over raw binary data buffers. Each typed array interprets the buffer as a sequence of a specific numeric type:

```
// Create a typed array with 4 entries (16 bytes total for Int32Array)
const int32 = new Int32Array(4);
int32[0] = 42;
int32[1] = 100;
int32[2] = 255;
int32[3] = 1024;
console.log(int32); // Int32Array [42, 100, 255, 1024]

// Float64Array (8 bytes per element)
const float64 = new Float64Array(3);
float64[0] = 3.14;
float64[1] = 2.718;
float64[2] = 1.618;
console.log(float64); // Float64Array [3.14, 2.718, 1.618]
```

ArrayBuffer and DataView

`ArrayBuffer` is a fixed-length raw binary buffer. `DataView` provides a flexible interface to read/write different types at any byte offset:

```
const buffer = new ArrayBuffer(16); // 16 bytes

const view = new DataView(buffer);
view.setInt32(0, 12345);   // Write 32-bit integer at byte 0
view.setFloat64(4, 3.1415); // Write 64-bit float at byte 4
view.setUint8(12, 255);    // Write unsigned byte at byte 12

console.log(view.getInt32(0));   // 12345
console.log(view.getFloat64(4)); // 3.1415
console.log(view.getUint8(12));  // 255

// Byte order (endianness)
console.log(view.getInt32(0, true)); // Little-endian read
```

Use Case: Typed arrays and ArrayBuffers are essential for WebGL graphics, audio processing (Web Audio API), file manipulation (FileReader), and network protocols (WebSockets). They provide fine-grained control over binary data with predictable performance.

Exercise: Create a Proxy that validates property values on an object. The proxy should ensure that any numeric property is between 0 and 100, and any string property is at most 20 characters long. Throw a `TypeError` with a descriptive message if validation fails. Create a sample object and test both valid and invalid assignments.

CHAPTER 8

Asynchronous JavaScript

Asynchronous programming is essential for non-blocking operations like fetching data, reading files, or waiting for user input. JavaScript handles asynchrony through callbacks, promises, and async/await.

Synchronous vs Asynchronous

Synchronous code runs line by line, blocking further execution until each operation completes. Asynchronous code lets the program continue while waiting for slow operations:

```
// Synchronous — blocks until complete
console.log("Start");
const data = fetchDataSync(); // Blocks for 2 seconds
console.log(data);

// Asynchronous — does not block
console.log("Start");
fetchDataAsync((data) => {
  console.log(data); // Runs later
});
console.log("End"); // Runs immediately after "Start"
```

Callbacks

A callback is a function passed as an argument to another function, to be executed later:

```
function fetchUser(id, callback) {
  setTimeout(() => {
    callback({ id, name: "Alice" });
  }, 1000);
}

fetchUser(1, (user) => {
  console.log("User:", user.name);
});

console.log("Waiting for user...");
```

Callback Hell

Nesting callbacks creates deeply indented, hard-to-maintain code known as "callback hell":

```
fetchUser(1, (user) => {
  fetchPosts(user.id, (posts) => {
    fetchComments(posts[0].id, (comments) => {
      fetchLikes(comments[0].id, (likes) => {
        console.log(likes);
        // Deep nesting continues...
      });
    });
  });
});
```

> **Problem:** Callback hell makes code hard to read, debug, and maintain. Promises and async/await provide a much cleaner alternative.

Promises

A `Promise` represents a value that may be available now, later, or never. It has three states: `pending`, `fulfilled`, or `rejected`.

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  setTimeout(() => {
    if (success) {
      resolve("Operation succeeded!");
    } else {
      reject("Operation failed!");
    }
  }, 1000);
});

promise
  .then((result) => console.log(result))
  .catch((error) => console.error(error))
  .finally(() => console.log("Done"));
```

Chaining Promises

```
fetchUser(1)
  .then((user) => fetchPosts(user.id))
  .then((posts) => fetchComments(posts[0].id))
  .then((comments) => console.log(comments))
  .catch((error) => console.error("Error:", error));
```

Promise.all

Wait for all promises to resolve (or any to reject):

```
const p1 = fetchUser(1);
const p2 = fetchUser(2);
const p3 = fetchUser(3);

Promise.all([p1, p2, p3])
  .then((users) => console.log("All users:", users))
  .catch((err) => console.error("One failed:", err));
```

Promise.race

Resolve or reject as soon as the first promise settles:

```
const timeout = new Promise((_, reject) =>
  setTimeout(() => reject(new Error("Timeout")), 5000)
);

const data = fetch("https://api.example.com/data");
Promise.race([data, timeout])
  .then((result) => console.log(result))
  .catch((err) => console.error(err));
```

> **Additional Promise Combinators:** `Promise.allSettled` waits for all promises regardless of rejection, returning their outcomes. `Promise.any` resolves when the first promise fulfills (rejects only if all reject).

async / await

Introduced in ES2017, `async/await` makes asynchronous code look synchronous:

```
async function loadUserData(userId) {
  try {
    const user = await fetchUser(userId);
    const posts = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);
    return comments;
  } catch (error) {
    console.error("Failed to load user data:", error);
    throw error;
  }
}

// Usage
loadUserData(1)
  .then((comments) => console.log(comments))
  .catch((err) => console.error(err));
```

Async functions always return a Promise

```
async function greet(name) {
  return `Hello, ${name}!`;
}

greet("Alice").then(console.log); // "Hello, Alice!"

async function example() {
  // Parallel execution with await
  const [user1, user2] = await Promise.all([
    fetchUser(1),
    fetchUser(2)
  ]);
  console.log(user1, user2);
}
```

Error Handling in Async Functions

Always wrap `await` calls in `try/catch` blocks to handle rejections gracefully:

```
async function safeFetch(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.error(`Fetch failed for ${url}:`, error.message);
    return null; // Graceful fallback
  }
}
```

The Event Loop

The event loop is JavaScript's concurrency model. It continuously checks the call stack and callback queue:

- **Call Stack** — Executes synchronous code (LIFO)
- **Web APIs** — Browser-provided APIs (timers, fetch, DOM events)
- **Task Queue (Callback Queue)** — Holds callbacks from Web APIs
- **Microtask Queue** — Promise callbacks have priority over task queue

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve()
  .then(() => console.log("3"));

console.log("4");
// Output: 1, 4, 3, 2
// Microtask (Promise) runs before the next task (setTimeout)
```

Key Concept: Microtasks (Promise callbacks, `queueMicrotask`) execute before the next macrotask (setTimeout, setInterval, I/O). This means promises resolve before timers, even when the timer delay is 0ms.

Exercise: Rewrite the following callback-hell code using async/await with try/catch: simulate fetching a user (1s), then their orders (1s), then order details (1s) — each step depends on the previous result. Use `setTimeout` wrapped in a Promise to simulate each async operation. Also add a timeout race using `Promise.race` that rejects if the whole chain takes longer than 4 seconds.

CHAPTER 9

Modules & Tooling

Modern JavaScript development relies on modules to organise code and tooling to automate repetitive tasks. This chapter covers ES6 modules, dynamic imports, module bundlers, npm, and build tools.

ES6 Modules

ES6 introduced a native module system using `import` and `export` keywords. A module is simply a file that exports values for other files to consume.

Named Exports

You can export multiple values from a module by prefixing declarations with `export`.

```
// math.js
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
```

Default Export

Each module can have one default export, typically used for the main functionality.

```
// calculator.js
export default class Calculator {
  constructor() { this.result = 0; }
  add(n) { this.result += n; return this; }
  subtract(n) { this.result -= n; return this; }
  getResult() { return this.result; }
}
```

Importing

Named imports use curly braces; default imports do not.

```
// app.js
import Calculator from './calculator.js';
import { PI, add } from './math.js';

const calc = new Calculator();
calc.add(10).subtract(3);
console.log(calc.getResult()); // 7
console.log(`PI is ${PI}`);
```

> **Module Scope:** Variables declared in a module are scoped to that module and never leak to the global scope. You must explicitly export anything you want to share.

Dynamic Import()

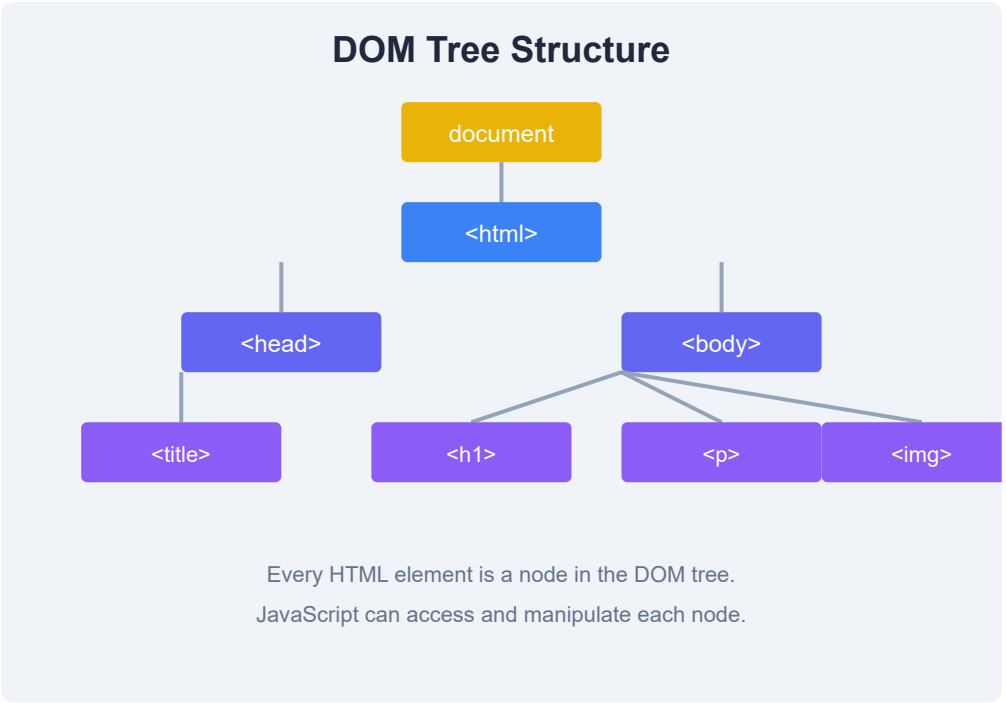
Use `import()` as a function to load modules on demand at runtime. It returns a promise.

```
const modulePath = './math.js';
const module = await import(modulePath);
console.log(module.PI);
```

Dynamic imports are useful for code splitting — loading only the code the user needs at the moment they need it.

Module Bundlers

Bundlers like **Webpack** and **Vite** take your module files and combine them into one or more bundle files optimised for the browser.



Webpack

Webpack is a mature bundler that processes your entry file, follows the dependency graph, and outputs bundled assets. It uses loaders for CSS, images, and transpilers.

```
// webpack.config.js (basic)
module.exports = {
  entry: './src/index.js',
  output: { path: __dirname + '/dist', filename: 'bundle.js' },
};
```

Vite

Vite is a next-generation build tool that leverages native ES modules during development for near-instant hot reloading. It uses Rollup under the hood for production builds.

```
# Create a new Vite project
npm create vite@latest my-app -- --template vanilla
```

npm Basics

npm (Node Package Manager) is the default package manager for Node.js. A `package.json` file tracks your project's metadata and dependencies.

```
{
  "name": "my-project",
  "version": "1.0.0",
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

```
# Installing packages
npm install lodash
npm install -D eslint   # dev dependency
```

Transpilers & Build Tools

Babel transpiles modern JavaScript (ES6+) into backwards-compatible versions for older browsers. It works as a plugin system and is commonly integrated with Webpack.

```
// .babelrc
{
  "presets": ["@babel/preset-env"]
}
```

> **Best Practice:** Always use a `.gitignore` file to exclude `node_modules/` and build output directories. Run `npm audit` regularly to check for vulnerable dependencies.

Exercise: Module Refactor

Take a script with three utility functions (e.g., `capitalize`, `reverse`, `randomInt`). Split them into a separate module file with named exports. Import them in your main file and use each function. Then create a default export class called `StringHelper` that wraps some of these utilities.

CHAPTER 10

Web APIs

Web APIs are interfaces provided by the browser that let JavaScript interact with the page, the device, and the network. This chapter surveys the most commonly used browser APIs and introduces JSON and jQuery.

Browser Web APIs Overview

Modern browsers ship dozens of APIs. Here are the most important ones for everyday development:

Geolocation API

Access the user's current geographic location (with permission).

```
navigator.geolocation.getCurrentPosition(pos => {
  console.log(`Lat: ${pos.coords.latitude}, Lng: ${pos.coords.longitude}`);
}, err => {
  console.error('Location denied:', err.message);
});
```

Web Storage: LocalStorage & SessionStorage

`localStorage` persists data across browser sessions; `sessionStorage` clears when the tab closes. Both store key-value pairs as strings.

```
// Save
localStorage.setItem('theme', 'dark');
// Read
const theme = localStorage.getItem('theme');
// Remove
localStorage.removeItem('theme');
// Clear all
localStorage.clear();
```

History API

Manipulate the browser's session history without triggering a full page reload — essential for single-page applications.

```
// Push a new state
history.pushState({ page: 1 }, 'Page 1', '/page1');
// Listen for back/forward
window.addEventListener('popstate', e => {
  console.log('State:', e.state);
});
```

Fetch API

Make HTTP requests from the browser. It returns a Promise that resolves to the `Response` object.

```
fetch('https://api.example.com/data')
  .then(res => { if (!res.ok) throw new Error('Network error'); return res.js
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Canvas API

Draw 2D graphics programmatically. The `<canvas>` element provides a drawing surface.

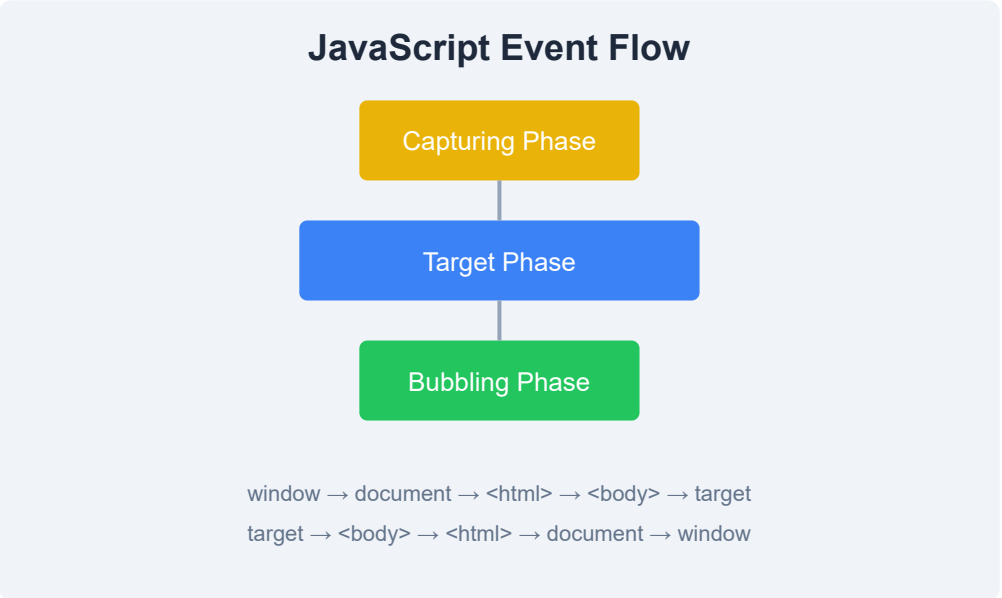
```
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');
ctx.fillStyle = 'red';
ctx.fillRect(10, 10, 100, 60);
```

Web Workers

Run JavaScript in a background thread, keeping the UI thread responsive.

```
// main.js
const worker = new Worker('worker.js');
worker.postMessage('Hello from main');
worker.onmessage = e => console.log('Worker says:', e.data);

// worker.js
onmessage = e => { postMessage('Hello back!'); };
```



AJAX Basics

AJAX (Asynchronous JavaScript and XML) lets web pages send and receive data from a server without refreshing. The modern approach uses the Fetch API; the older approach uses `XMLHttpRequest`.

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://api.example.com/data');
xhr.onload = () => { console.log(JSON.parse(xhr.responseText)); };
xhr.send();
```

JSON Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is language-independent but uses conventions familiar to JavaScript developers.

```
{
  "name": "Alice",
  "age": 30,
  "skills": ["JavaScript", "Python"],
  "active": true
}
```

JSON.parse() & JSON.stringify()

```
const jsonStr = '{"name":"Bob","age":25}';
const obj = JSON.parse(jsonStr);
console.log(obj.name); // Bob

const backToJson = JSON.stringify(obj);
console.log(backToJson); // {"name":"Bob","age":25}
```

jQuery Introduction

jQuery is a fast, small library that simplifies HTML document traversal, event handling, and animation. While modern frameworks have reduced its necessity, many legacy projects still use it.

Selectors

```
// jQuery selectors (using $ alias)
$('.my-class').hide();
$('#my-id').css('color', 'red');
$('p:first').text('First paragraph');
```

Events

```
$('button').click(() => alert('Clicked!'));
$('form').on('submit', e => {
  e.preventDefault();
  console.log('Form submitted');
});
```

AJAX with jQuery

```
$.ajax({
  url: '/api/data',
  method: 'GET',
  success: data => console.log(data),
  error: (xhr, status, err) => console.error(err)
});
```

> **Modern alternative:** Consider using vanilla JavaScript with the Fetch API instead of jQuery for new projects. It reduces bundle size and leverages native browser capabilities.

Exercise: LocalStorage Theme Switcher

Build a page with a button that toggles between light and dark themes. Store the user's preference in `localStorage` so the theme persists across page reloads. Use the Geolocation API to display the user's approximate latitude and longitude below the theme toggle.

CHAPTER 11

AJAX & JSON

AJAX and JSON together form the backbone of modern web communication. This chapter dives deep into making HTTP requests from the browser, handling responses, and working with the JSON data format.

XMLHttpRequest (The Old Way)

`XMLHttpRequest` is the original browser API for making HTTP requests. It is event-driven and supports both synchronous and asynchronous modes.

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1');
xhr.onload = function() {
  if (xhr.status >= 200 && xhr.status < 300) {
    const data = JSON.parse(xhr.responseText);
    console.log('Title:', data.title);
  } else {
    console.error('Request failed:', xhr.status);
  }
};
xhr.onerror = () => console.error('Network error');
xhr.send();
```

> **Caveats:** `XMLHttpRequest` does not support promises natively, making it verbose. It also ignores the `Content-Type` when using `overrideMimeType()`. Modern code should prefer the Fetch API.

Fetch API (Modern)

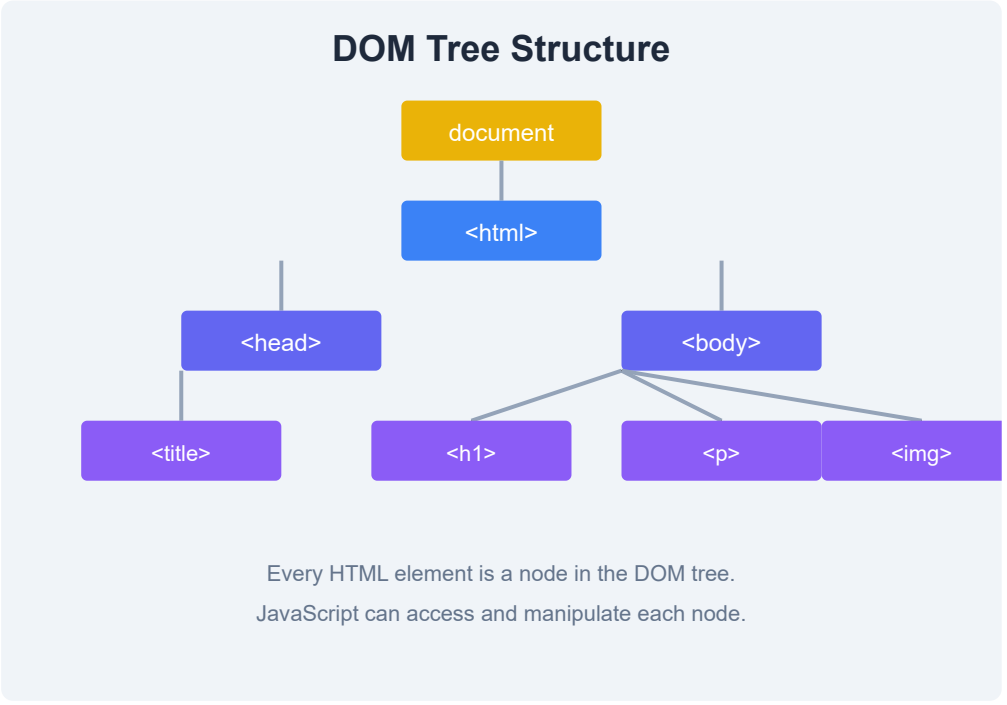
The Fetch API provides a cleaner, promise-based interface for making HTTP requests. It is built into all modern browsers.

GET Request

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(response => {
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  })
  .then(posts => console.log('Posts:', posts))
  .catch(err => console.error('Fetch error:', err));
```

POST Request

```
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    title: 'New Post',
    body: 'This is the content.',
    userId: 1
  })
})
  .then(res => res.json())
  .then(data => console.log('Created:', data))
  .catch(err => console.error(err));
```



Handling Responses

The `Response` object provides several methods to read the body in different formats:

```
fetch('/data')
  .then(res => {
    console.log('Status:', res.status);
    console.log('Headers:', res.headers.get('Content-Type'));
    return res.text(); // or .json(), .blob(), .formData()
  })
  .then(data => console.log(data));
```

Error Handling Strategies

Network failures and non-2xx status codes must be handled explicitly.

```
async function fetchData(url) {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`Status ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error('Failed to fetch:', err.message);
    return null;
  }
}
```

JSON Structure & Types

JSON supports these data types: string, number, boolean, null, object, and array. All keys and string values must be enclosed in double quotes.

```
{
  "string": "Hello",
  "number": 42,
  "boolean": true,
  "nullValue": null,
  "array": [1, 2, 3],
  "object": { "nested": "value" }
}
```

JSON vs XML

JSON is more concise, easier to parse, and maps directly to JavaScript objects. XML is more verbose but supports attributes, namespaces, and schema validation.

```
<!-- XML version of the same data -->
<person>
  <name>Alice</name>
  <age>30</age>
</person>

// JSON equivalent
{"name": "Alice", "age": 30}
```

> **Tip:** Use `JSON.stringify(value, replacer, space)` to pretty-print JSON with indentation. The `space` parameter controls the indentation width.

Working with REST APIs

REST APIs expose resources via standard HTTP methods. A typical workflow involves fetching a list, creating a new resource, updating one, and deleting it.

```
const BASE = 'https://jsonplaceholder.typicode.com';

async function apiDemo() {
  // GET all
  const todos = await fetch(`${BASE}/todos?_limit=3`).then(r => r.json());
  console.log('Todos:', todos);

  // POST create
  const created = await fetch(`${BASE}/todos`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: 'Learn Fetch', completed: false })
  }).then(r => r.json());
  console.log('Created:', created);
}
```

Async/Await with Fetch

Async/await makes asynchronous fetch code read like synchronous code, improving readability.

```
async function getUser(id) {
  const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`)
  if (!res.ok) throw new Error('User not found');
  return await res.json();
}

(async () => {
  try {
    const user = await getUser(1);
    console.log(user.name);
  } catch (err) {
    console.error(err);
  }
})();
```

Exercise: GitHub User Search

Create a page with an input field and a button. When the user types a GitHub username and clicks the button, use the Fetch API to call `https://api.github.com/users/{username}`. Display the user's avatar, name, bio, and public repository count. Handle errors gracefully if the user is not found.

CHAPTER 12

Graphics

JavaScript provides two primary ways to create graphics in the browser: Canvas (pixel-based, imperative) and SVG (vector-based, declarative). This chapter covers both with practical drawing examples.

HTML Canvas

The `<canvas>` element provides a bitmap drawing surface. You access its 2D rendering context through JavaScript and issue drawing commands.

Getting the Context

```
<canvas id="myCanvas" width="400" height="300"></canvas>
<script>
  const canvas = document.getElementById('myCanvas');
  const ctx = canvas.getContext('2d');
</script>
```

Drawing Shapes

```
// Rectangle
ctx.fillStyle = '#3b82f6';
ctx.fillRect(20, 20, 150, 100);
ctx.strokeStyle = '#1e293b';
ctx.lineWidth = 3;
ctx.strokeRect(20, 20, 150, 100);

// Circle (arc)
ctx.beginPath();
ctx.arc(300, 70, 40, 0, Math.PI * 2);
ctx.fillStyle = '#22c55e';
ctx.fill();
ctx.stroke();
```

Paths

```
ctx.beginPath();
ctx.moveTo(50, 200);
ctx.lineTo(200, 250);
ctx.lineTo(100, 280);
ctx.closePath();
ctx.fillStyle = '#eab308';
ctx.fill();
```

Text

```
ctx.font = 'bold 24px Arial';
ctx.fillStyle = '#0f172a';
ctx.fillText('Hello Canvas!', 50, 50);
ctx.strokeText('Hello Canvas!', 50, 90);
```

Colors & Gradients

```
const gradient = ctx.createLinearGradient(0, 0, 400, 0);
gradient.addColorStop(0, '#ff0000');
gradient.addColorStop(1, '#0000ff');
ctx.fillStyle = gradient;
ctx.fillRect(0, 200, 400, 50);
```

SVG (Scalable Vector Graphics)

SVG describes 2D graphics using XML markup. Elements are part of the DOM and can be styled with CSS and manipulated with JavaScript.

Inline SVG Shapes

```
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="80" height="80" fill="#3b82f6" rx="8" />
  <circle cx="150" cy="50" r="40" fill="#22c55e" />
  <polygon points="100,150 50,190 150,190" fill="#eab308" />
</svg>
```

SVG Groups & Transforms

```
<svg width="300" height="200" xmlns="http://www.w3.org/2000/svg">
  <g transform="translate(20, 20)">
    <rect width="100" height="60" fill="#f59e0b" />
    <text x="50" y="35" text-anchor="middle" fill="white">Group</text>
  </g>
  <g transform="rotate(45, 200, 80)">
    <rect x="150" y="40" width="80" height="80" fill="#ef4444" />
  </g>
</svg>
```

Drawing Examples

Bar Chart (Canvas)

```
const data = [30, 85, 55, 70, 45];
const barWidth = 50;
const gap = 20;
ctx.fillStyle = '#3b82f6';
data.forEach((val, i) => {
  const x = 30 + i * (barWidth + gap);
  const h = val * 2;
  ctx.fillRect(x, 250 - h, barWidth, h);
});
```

Smiley Face (Canvas)

```
// Face
ctx.beginPath();
ctx.arc(200, 150, 80, 0, Math.PI * 2);
ctx.fillStyle = '#fef9c3';
ctx.fill();
ctx.stroke();
// Eyes
ctx.fillStyle = '#1e293b';
ctx.beginPath(); ctx.arc(170, 130, 8, 0, Math.PI * 2); ctx.fill();
ctx.beginPath(); ctx.arc(230, 130, 8, 0, Math.PI * 2); ctx.fill();
// Smile
ctx.beginPath();
ctx.arc(200, 150, 50, 0.1 * Math.PI, 0.9 * Math.PI);
ctx.stroke();
```

Clock (Canvas)

```
function drawClock() {
  const now = new Date();
  const sec = now.getSeconds() * (Math.PI / 30);
  const min = now.getMinutes() * (Math.PI / 30);
  const hr = now.getHours() * (Math.PI / 6);

  ctx.clearRect(0, 0, 400, 400);
  ctx.beginPath();
  ctx.arc(200, 200, 150, 0, Math.PI * 2);
  ctx.stroke();

  // Hour hand, minute hand, second hand
  ctx.beginPath(); ctx.moveTo(200, 200); ctx.lineTo(200 + 80 * Math.sin(hr),
  ctx.beginPath(); ctx.moveTo(200, 200); ctx.lineTo(200 + 120 * Math.sin(min)
  ctx.beginPath(); ctx.moveTo(200, 200); ctx.lineTo(200 + 140 * Math.sin(sec)
}
```

Canvas vs SVG Comparison

Aspect	Canvas	SVG
Rendering	Pixel-based	Vector-based
DOM integration	None	Full DOM tree
Performance	Fast for many elements	Best for fewer, interactive elements
Accessibility	Not accessible	Accessible via ARIA
Use case	Games, image processing	Diagrams, icons, charts

Exercise: Animated Bouncing Ball

Use the Canvas API to create a bouncing ball animation. The ball should start at the top-left corner and bounce off the edges of the canvas. Use `requestAnimationFrame()` for smooth animation. Add a second ball with a different colour and speed.

CHAPTER 13

Projects

Building real projects is the best way to solidify your JavaScript skills. This chapter presents five mini-project ideas and walks through the development of a complete to-do list application.

Mini Project Ideas

1. To-Do List

A classic CRUD application. Users can add, complete, and delete tasks. Persist tasks with `localStorage`. Filter by all / active / completed.

2. Calculator

Build a four-function calculator with a clean UI. Handle chained operations, decimal points, and keyboard input. Display the current expression and result.

3. Weather App

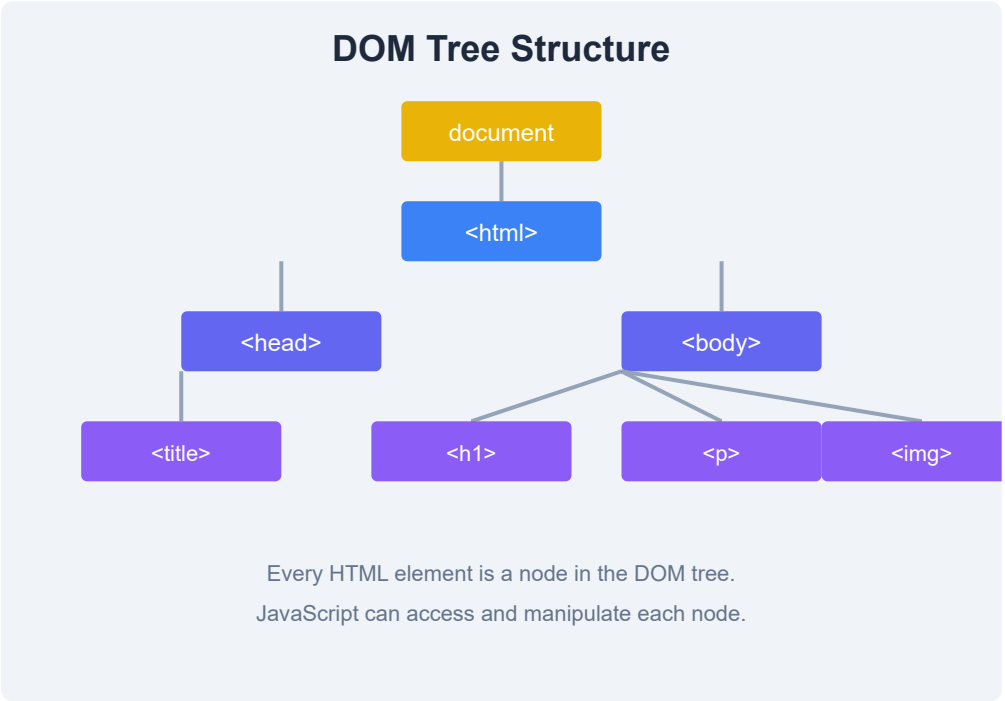
Use the Fetch API to call a free weather API (e.g., OpenWeatherMap). Show temperature, humidity, conditions, and a forecast. Add an input field to search by city name.

4. Quiz Game

A multiple-choice quiz that tracks the score and shows progress. Fetch questions from an API or use a static array. Include a timer and a final results screen.

5. Countdown Timer

Let users set a timer with hours, minutes, and seconds. Display a countdown that updates every second. Play a sound or show an alert when time runs out.



Walkthrough: To-Do List App

We will build a complete to-do list step by step. The final app will use `localStorage` for persistence and include add / delete / toggle-complete functionality.

HTML Structure

```
<div id="app">
  <h1>To-Do List</h1>
  <input id="taskInput" type="text" placeholder="Add a task...">
  <button id="addBtn">Add</button>
  <ul id="taskList"></ul>
  <div id="filters">
    <button data-filter="all" class="active">All</button>
    <button data-filter="active">Active</button>
    <button data-filter="completed">Completed</button>
  </div>
</div>
```

JavaScript Logic

```
const taskInput = document.getElementById('taskInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');

let tasks = JSON.parse(localStorage.getItem('tasks')) || [];
let filter = 'all';

function saveAndRender() {
  localStorage.setItem('tasks', JSON.stringify(tasks));
  render();
}

function render() {
  const filtered = tasks.filter(t => {
    if (filter === 'active') return !t.completed;
    if (filter === 'completed') return t.completed;
    return true;
  });

  taskList.innerHTML = filtered.map((t, i) => `
    <li class="${t.completed ? 'done' : ''}">
      <input type="checkbox" ${t.completed ? 'checked' : ''} data-index="${i}
      <span>${t.text}</span>
      <button data-index="${i}" class="delete">&times;</button>
    </li>
  `).join('');
}

addBtn.addEventListener('click', () => {
  const text = taskInput.value.trim();
  if (text) {
    tasks.push({ text, completed: false });
    taskInput.value = '';
    saveAndRender();
  }
});

taskList.addEventListener('click', e => {
  if (e.target.classList.contains('delete')) {
    tasks.splice(+e.target.dataset.index, 1);
    saveAndRender();
  }
```

```
});

taskList.addEventListener('change', e => {
  if (e.target.type === 'checkbox') {
    tasks[+e.target.dataset.index].completed = e.target.checked;
    saveAndRender();
  }
});

document.getElementById('filters').addEventListener('click', e => {
  if (e.target.dataset.filter) {
    filter = e.target.dataset.filter;
    document.querySelectorAll('#filters button').forEach(b => b.classList.rem
    e.target.classList.add('active');
    render();
  }
});

render();
```

> **Tip:** Use `data-*` attributes to store metadata directly in HTML elements. They are accessed via the `dataset` property and are valid HTML5.

Using Online Code Editors

Platforms like **CodePen**, **JSFiddle**, and **CodeSandbox** let you write HTML, CSS, and JavaScript in the browser and see results instantly. They are great for prototyping, sharing snippets, and experimenting.

Project Structure Tips

- **Separate concerns:** Keep HTML, CSS, and JavaScript in separate files.
- **Use meaningful names:** Name variables and functions by what they do.
- **Comment sparingly:** Write self-documenting code; use comments only for complex logic.
- **Version control:** Initialise a Git repository from day one.
- **Test incrementally:** Verify each feature works before adding the next.

Exercise: Build the Quiz Game

Implement the quiz game project. Create an array of at least 5 question objects with `question`, `options` (array of 4 strings), and `correctAnswer` (index). Display one question at a time, highlight the correct/wrong answer after selection, move to the next question, and display the final score. Add a restart button.

CHAPTER 14

Practice & Certification

Practice is the bridge between learning and mastery. This chapter provides quiz questions, coding challenges, a study plan, interview preparation tips, and certification paths to guide your next steps.

JavaScript Quiz Questions

Test your knowledge with these fundamental questions:

1. **What is the difference between `==` and `===`?**
   `==` compares values after type coercion; `===` compares both value and type without coercion.
2. **What does `let` vs `const` vs `var` mean?**
   `var` is function-scoped; `let` and `const` are block-scoped. `const` prevents reassignment.
3. **How does prototypal inheritance work?**
   Objects inherit properties from other objects via the prototype chain. `Object.create()` and `class` syntax both use prototypes.
4. **Explain the event delegation pattern.**
   Attach a single event listener to a parent instead of many listeners to each child. Use `e.target` to determine which child triggered the event.
5. **What is a closure?**
   A function that retains access to its outer scope even after the outer function has returned.

Exercises by Difficulty

Beginner

- Write a function that reverses a string.
- Write a function that checks if a string is a palindrome.
- Build a Fahrenheit-to-Celsius converter.

- Print the numbers 1–100. For multiples of 3 print “Fizz”, for 5 print “Buzz”, for both print “FizzBuzz”.

Intermediate

- Implement `Array.prototype.map()` from scratch.
- Write a deep clone function that handles nested objects and arrays.
- Build a promise-based rate limiter that queues calls.
- Create a debounce function with leading/trailing options.

Advanced

- Write a reactive state management system (like a minimal Redux).
- Implement a virtual DOM diffing algorithm.
- Build a simple Promise.all() polyfill.
- Create a template engine that parses {{ mustache }} syntax.

Bootcamp-Style Curriculum

If you prefer a structured approach, here is an 8-week study plan:

Week	Topics
1	Variables, data types, operators, conditionals
2	Loops, functions, arrays, objects
3	DOM manipulation, events, forms
4	ES6 features, classes, modules
5	Asynchronous JS: callbacks, promises, async/await
6	AJAX, Fetch API, JSON, REST APIs
7	Canvas, SVG, and project work
8	Review, interview prep, portfolio building

Interview Preparation

Common interview topics include: closures, hoisting, the event loop, `this` binding, prototypal inheritance, promises, scope, and performance optimisation. Practice explaining these concepts aloud and writing solutions on a whiteboard or plain text editor.

```
// Common interview question: Flatten a nested array
function flatten(arr) {
  return arr.reduce((acc, item) =>
    acc.concat(Array.isArray(item) ? flatten(item) : item), []);
}
console.log(flatten([1, [2, [3, [4]]]])); // [1, 2, 3, 4]
```

> **Tip:** Understand the event loop, microtasks vs macrotasks, and how `setTimeout(fn, 0)` defers execution. This is one of the most frequently tested concepts in JavaScript interviews.

Certification Paths

- **freeCodeCamp** — Free, self-paced JavaScript Algorithms and Data Structures certification with hundreds of coding challenges.
- **MDN Web Docs** — Comprehensive reference and learning area with tutorials, guides, and interactive examples.
- **Microsoft Learn** — Free learning paths for JavaScript, TypeScript, and related web technologies.
- **W3Schools** — Beginner-friendly tutorials with a "Try it Yourself" editor and a JavaScript certification exam.

Exercise: Build Your Portfolio

Create a single-page portfolio site that showcases three JavaScript projects you have built. Include a project title, a brief description, a screenshot or live demo link, and a link to the source code on GitHub. Style it with CSS and make it responsive.

CHAPTER 15

References

This chapter is a quick-reference guide to JavaScript's core types, operators, built-in objects, and methods. Use it as a cheat sheet when writing code or preparing for interviews.

Core Data Types

Type	Example	typeof
Number	`42`, `3.14`	`"number"`
String	`"hello"`, `'world'`	`"string"`
Boolean	`true`, `false`	`"boolean"`
Undefined	`undefined`	`"undefined"`
Null	`null`	`"object"`
BigInt	`9007199254740991n`	`"bigint"`
Symbol	`Symbol("id")`	`"symbol"`
Object	`{a:1}`, `[1,2]`	`"object"`

Operators Reference

Category	Operators
Arithmetic	`+ - * / % ** ++ --`
Assignment	`= += -= *= /= %= **=`
Comparison	`== === != !== > < >= <=`
Logical	`&& || ! ??`
Bitwise	`& | ^ ~ << >> >>>`
Ternary	`condition ? expr1 : expr2`
Spread / Rest	`...arr`

Class Syntax Reference

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound.`); }
  static classify() { return 'Mammal'; }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { console.log(`${this.name} barks.`); }
}
```

Array Methods

```
// Iteration
arr.forEach(fn); arr.map(fn); arr.filter(fn); arr.reduce(fn, init); arr.some(

// Mutation
arr.push(v); arr.pop(); arr.shift(); arr.unshift(v); arr.splice(i, n); arr.so

// Access / search
arr.concat(arr2); arr.slice(i, j); arr.indexOf(v); arr.find(fn); arr.findInde

// Static
Array.from(iterable); Array.of(v1, v2, ...); Array.isArray(v);
```

String Methods

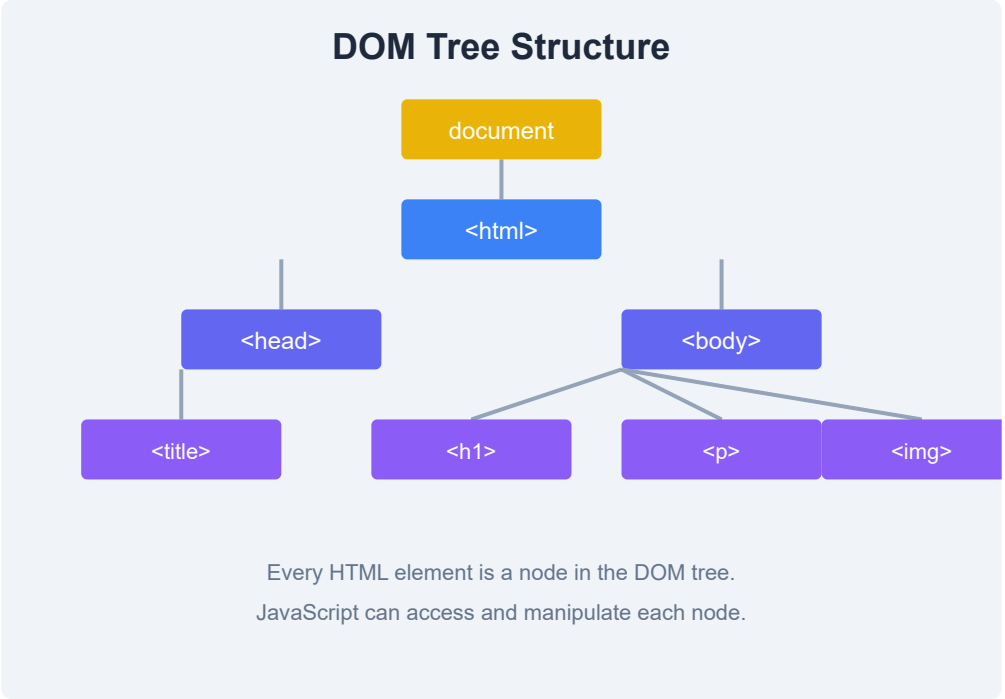
```
"hello".toUpperCase(); "HELLO".toLowerCase();
"hello".trim(); "hello".charAt(0); "hello".indexOf("e");
"hello".slice(1, 4); "hello".split(""); "hello".includes("ell");
"hello".startsWith("he"); "hello".endsWith("lo");
"hello".replace("l", "x"); "hello".repeat(3);
"a,b,c".split(","); "hello".padStart(10, "*");
```

Number & Math Methods

```
Number.isNaN(v); Number.isInteger(v); Number.parseFloat("3.14"); Number.parse
Number(v).toFixed(2);
Math.round(3.7); Math.floor(3.7); Math.ceil(3.2);
Math.min(1, 5, 3); Math.max(1, 5, 3);
Math.random(); // 0 to < 1
Math.abs(-5); Math.pow(2, 3); Math.sqrt(16); Math.PI;
```

Date Methods

```
const d = new Date();
d.getFullYear(); d.getMonth(); d.getDate(); d.getDay();
d.getHours(); d.getMinutes(); d.getSeconds();
d.toISOString(); d.toLocaleDateString(); d.toLocaleTimeString();
// Setting
d.setFullYear(2025); d.setMonth(11); d.setDate(25);
// Timestamp
Date.now(); d.getTime();
```



Browser Compatibility

All examples in this tutorial target modern browsers (Chrome, Firefox, Safari, Edge). For older browser support, use caniuse.com to check feature availability and consider transpilers like Babel and polyfills like `core-js`.

> **Keep learning:** The MDN Web Docs (developer.mozilla.org) is the most authoritative JavaScript reference. Bookmark it and consult it regularly.

Exercise: Build Your Own Cheat Sheet

Create an interactive HTML page that displays all Array methods. For each method, show its syntax, a short description, and a clickable “Run Example” button that logs the result to the console. Use the `data-*` attributes to store the method name and parameters.